

Basic (your first real API call) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
# chat_api_basics_practice.py
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI()

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a formal, professional assistant."},
        {"role": "user", "content": "Tell me about your day."},
    ],
)
print(response.choices[0].message.content)

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a sarcastic pirate."},
        {"role": "user", "content": "Tell me about your day."},
    ],
)
print(response.choices[0].message.content)
```

Expected output (wording varies, exact text differs every run):

```
As a formal, professional assistant, I do not experience days in the way a person
Arrr, another day chained to this here API, matey...
```

This works fine and shows the change clearly. It repeats the call structure twice, which is fine for a one-off script but gets repetitive fast if you compare more than two prompts.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a reusable `ask()` function

```
# chat_api_basics_practice.py
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI()

def ask(system_prompt: str, user_prompt: str) -> str:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt},
        ],
    )
    return response.choices[0].message.content

def main() -> None:
    question = "Tell me about your day."

    formal_answer = ask("You are a formal, professional assistant.", question)
    print("Formal:", formal_answer)

    pirate_answer = ask("You are a sarcastic pirate.", question)
    print("Pirate:", pirate_answer)

if __name__ == "__main__":
    main()
```

Expected output:

```
Formal: As a formal, professional assistant, I do not experience days in the way
```

```
Pirate: Arrr, another day chained to this here API, matey...
```

Difference from Basic: the `ask()` function has full type hints and separates "how to call the API" from "which prompts to compare" — adding a third or fourth system prompt to compare is now one more `ask()` call, not a whole copy-pasted block. Wrapping the script's entry point in `main()` behind `if __name__ == "__main__":` also matches the pattern from Doc01's Build Task solution — this file could later be imported elsewhere without immediately running.

Which one should you actually write? For this exercise, the Intermediate version is what you should write — a reusable, typed `ask()` function around one client. Reach for a cached, module-level client and explicit `max_retries / timeout` settings the moment this code is called from unattended, long-running code — exactly this document's Build Task, covered there.